UniConnect - MVP Use Cases

Team: FRAZER

Project: UniConnect - Academic Communication Platform

1. MVP Feature List & User Goals

The Minimum Viable Product (MVP) covers the following core functionalities:

1. **User Authentication & RBAC:** Users log in to access role-specific features (Student, Delegate, Professor).
2. **Group Messaging:** Users communicate in real-time within their class groups.
3. **Official Announcements:** Professors and Delegates post priority updates for the class.
4. **Study Event Organization:** Students schedule revision sessions for their peers.
5. **Delegate Election System:** Students vote securely for their class representatives.
6. **Class Group Management:** Professors create class environments and manage rosters.
7. **Timetable Access:** Centralized view of official courses and student events.

2. Detailed Use Cases

UC-01 Log in to System

Primary actor: Registered User (Student, Delegate, or Professor)

Goal: Securely access the platform to view private class groups and role-specific dashboards.

Trigger: User opens the application URL or clicks "Login" on the landing page.

Preconditions:

- The user has a registered account in the database.
- The system is online and accessible.

Postconditions:

- **Success:** A valid session token (JWT) is stored in the client. The user is redirected to the Dashboard matching their role.
- **Failure:** User remains on the Login screen with an error message.

Main success path:

1. User navigates to the Login Screen.
2. User enters a valid university email address.
3. User enters the correct password.
4. User clicks the "Sign In" button.
5. System validates credentials and determines the user's role (e.g., Delegate).
6. System generates an access token including the role claim.
7. Client stores the token and redirects to the Dashboard.

Alternative flows:

- **A1 (User Mistake - Invalid Password):** In step 5, if the password does not match the database hash, the system returns a 401 error. The UI displays "Incorrect email or password" and clears the password field.
- **A2 (System Issue - Database Timeout):** In step 4, if the database is unreachable, the system catches the connection error. The UI displays "System unavailable. Please try again later."

Acceptance criteria:

- Valid credentials redirect to the dashboard within 2 seconds.
- Invalid credentials show a clear, red error message without page reload.
- The user's role is correctly identified upon login.
- **Related screens:** Login Screen, Dashboard.
- **Related data objects:** User {id, email, password_hash, role}.

UC-02 Send Class Group Message

Primary actor: Student (Enrolled in the class)

Goal: Share information or ask a question within a specific class context.

Trigger: User types in the chat input field within a Class Feed.

Preconditions:

- User is logged in and is a member of the target class.
- The Class Feed screen is open.

Postconditions:

- A new Message record is saved in the database.
- The message appears at the bottom of the feed for all class members.

Main success path:

1. System loads the message history for the class.
2. User clicks the message input field.
3. User types: "Does anyone have the notes from today?".
4. User clicks the "Send" button.
5. System validates the message is not empty.
6. System saves the message to the database linked to the user and class.
7. UI updates the chat view to show the new message immediately via WebSocket.

Alternative flows:

- **A1 (User Mistake - Empty Message):** User tries to click "Send" with an empty input. The button is disabled.
- **A2 (System Issue - Network Fail):** If the internet connection drops during step 5, the request fails. The UI shows a "Failed to send" icon.

Acceptance criteria:

- Message appears in the feed immediately.
- Sender's name and timestamp are visible.
- User cannot send a message to a class they are not enrolled in.
- **Related screens:** Class Detail/Feed Screen.
- **Related data objects:** Message {id, content, sender_id, class_id, timestamp}.

UC-03 Post Official Announcement

Primary actor: Class Delegate (or Professor)

Goal: Pin important information (e.g., "Exam Cancelled") to the top of the class feed.

Trigger: User clicks the "New Announcement" button in the Announcement tab.

Preconditions:

- User has the role **DELEGATE** or **PROFESSOR**.
- User is an admin/delegate of the specific class.

Postconditions:

- A new Announcement record is created.
- The announcement appears in the dedicated "Official" section.

Main success path:

1. User navigates to the announcement tab.
2. User clicks the "Create Announcement" button.
3. User enters a Title ("Exam Rescheduled") and Body Content.
4. User selects "High Priority" checkbox.
5. User clicks "Publish".
6. System verifies the user's permissions (Role check).
7. System saves the announcement.
8. UI displays the new announcement at the top of the feed.

Alternative flows:

- **A1 (User Mistake - Missing Title):** User clicks "Publish" without a title. System highlights the Title field in red.
- **A2 (System Issue - API Error):** If the server returns a 500 error, the UI keeps the modal open to prevent data loss.

Acceptance criteria:

- Only Delegates/Professors see the creation button.
- Announcements are visually distinct (e.g., different background color/badge).
- **Related screens:** Class Feed Screen, Create Announcement Modal.
- **Related data objects:** Announcement {id, title, content, priority, author_id}.

UC-04 Create Study Event

Primary actor: Student

Goal: Organize a revision session or group study meeting for the class.

Trigger: User clicks "Event planning" in the sidebar.

Preconditions:

- User is enrolled in the class.

Postconditions:

- A new StudyEvent record is created.
- The event is listed in the class schedule/calendar.

Main success path:

1. User navigates to the "Events planning" tab.

2. User enters details: "Maths Revision", Date, Time, and Location ("Library Room 3").
3. User clicks "Create Event".
4. System validates that the date is in the future.
5. System saves the event.

Alternative flows:

- **A1 (User Mistake - Past Date):** User selects a date in the past. Validation fails.
- **A2 (System Issue - Data Conflict):** If the system detects a duplicate event ID, it returns an error.

Acceptance criteria:

- Event appears in the list tabletable tab.
- Location field is mandatory.
- **Related screens:** Events List Screen, Event Creation Form.
- **Related data objects:** StudyEvent {id, title, date, location, organizer_id}.

UC-05 Vote for Class Delegate

Primary actor: Student

Goal: Cast a vote for a representative in an active class election.

Trigger: User clicks on the "Vote Now" banner or "Elections" tab.

Preconditions:

- An election is currently active.
- The user has **not yet voted** in this specific election.

Postconditions:

- A Vote record is created anonymously.
- The user is marked as "Voted" (UI locked).

Main success path:

1. User clicks the "Elections" tab.
2. System displays active elections and list of Candidates.
3. User selects the radio button for "Candidate A".
4. User clicks "Submit Vote".
5. System displays a confirmation dialog.

6. User confirms the action.
7. System checks the Vote table constraint to ensure unique vote.
8. System records the vote.
9. UI triggers **Multimedia Success Animation** (Confetti) and hides the ballot.

Alternative flows:

- **A1 (User Mistake - No Selection):** User clicks "Submit" without selecting a candidate. UI displays error.
- **A2 (System Issue - Double Vote):** Backend rejects the request with "403 Forbidden - Already Voted".

Acceptance criteria:

- User can only submit one vote per election.
- Voting interface is disabled immediately after submission.
- **Related screens:** Election Tab, Voting Modal.
- **Related data objects:** Election {id, is_active}, Vote {election_id, candidate_id}.

UC-06 Create and Manage Class Group

Primary actor: Professor (Admin)

Goal: Establish a new class environment and manage the student roster.

Trigger: Professor clicks "Create Group" in the Admin Panel.

Preconditions:

- User is logged in with the role **PROFESSOR**.

Postconditions:

- A new Group record is created.
- Selected students are linked to the group.

Main success path:

1. Professor clicks "Create Group" in the sidebar.
2. Professor enters the group name (e.g., "Software Engineering L2").
3. Professor adds students by email or selects from a global list.
4. Professor clicks "Save Group".
5. System validates that the group name is unique.
6. System creates the group and redirects to the Group Dashboard.

Alternative flows:

- **A1 (Validation Error):** Group name is empty or duplicate.
- **A2 (Permission Error):** A student tries to access this API endpoint (403 Forbidden).

Acceptance criteria:

- Only Professors can access this feature.
- The new group appears immediately in the sidebar.
- **Related screens:** Admin Panel.
- **Related data objects:** Group {id, name}, UserGroup {user_id, group_id}.

UC-07 View Class Timetable

Primary actor: Student (or Professor)

Goal: Consult the official weekly schedule and integrated study events.

Trigger: User clicks the "Timetables" tab.

Preconditions:

- User is enrolled in the class.

Postconditions:

- System displays a 5-day grid with merged data.

Main success path:

1. User clicks the "Timetables" icon.
2. System fetches official course data for the group.
3. System fetches active revision sessions (StudyEvents).
4. UI renders a weekly grid (Mon-Fri).
5. Official courses appear in Lavender; Study events in Mint Green.
6. User clicks an event to see location details.

Alternative flows:

- **A1 (Empty Schedule):** UI displays "No schedule uploaded yet."

Acceptance criteria:

- Grid is responsive.
- Events do not overlap visually.
- **Related screens:** Timetable View.

- **Related data objects:** Group {schedule_path}, StudyEvent {title, date, location}.